Comparing Import Graphs and Issue Text for Coding Agent Context

NEIL LUNAVAT, Pimpri Chinchwad College of Engineering, India
MEHZABIN PATHAN, Pimpri Chinchwad College of Engineering, India

An LLM-based coding agent has a finite context window: before editing a file in an unfamiliar repository, it must decide what else to read. Given the file being changed, this article compares the two signals tools use for this choice, proximity in the import graph and the issue text, and their fusion, on 2,412 change tasks drawn from 1,162 pull requests in 112 Python repositories.

The main contribution of this article is a measurement protocol. Benchmarks built from issue reports mix two regimes: in two tasks in five, the issue names a file the change touches. Stratifying tasks by such naming, comparing each ordering with an issue-free twin, and deleting only the names from the same issue separate the two. The issue contributes +0.156 AUC to the best ordering when it names a key file, compared with a non-significant +0.016 otherwise, and approximately half of this difference is the naming itself. Under these controls, graph and text signals stop competing, and a human-annotated key agrees with the patch-derived key. Our recommendation, fusing a walk from the file with issue retrieval, held under predictions committed before scoring on seven unseen repositories and on the human-annotated key.

CCS Concepts: • **Software and its engineering** → **Software maintenance tools**; • **Information systems** → *Information retrieval*; • **General and reference** → *Empirical studies.*

Additional Key Words and Phrases: LLM agents, context windows, code search, context retrieval, dependency graphs, benchmark validity, empirical software engineering

ACM Reference Format:
Neil Lunavat and Mehzabin Pathan. 2026. Comparing Import Graphs and Issue Text for Coding Agent Context. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (October 2026), 25 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Consider an LLM coding agent that is given a bug report and a file to change. Before it can produce any edit, it must decide which additional content to place in its context window, and including everything is not an option. In the corpus we study, the median pull request targets a repository of 286,385 tokens (counting the Python files in the import graph and excluding tests and non-Python files), and 74.8% of pull requests target a repository larger than a 128,000-token window. The files that contain the answer account for a median of 2.98% of the repository. A selection step is therefore unavoidable.

Two signals are available for this selection. The dominant one in tool development is the dependency graph: Aider constructs a repository map from definitions and references, and RepoGraph, LocAgent and CoSIL traverse a code graph outward from a starting point, based on the premise that the information needed is located near the code being changed. The other signal is the issue report, which SWE-bench's own retrieval baseline queries with BM25 [10] and which every agent reads. In this article, we compare the two signals, and their fusion, as *reading orders* evaluated by how much of an answer key they cover per line read. In every task, the reader is given the file being changed, and the question is which *other* files to read. Our focus is therefore on retrieval rather than repair.

Authors' addresses: Neil Lunavat, Pimpri Chinchwad College of Engineering, India, neil.lunavat24@pccoepune.org; Mehzabin Pathan, Pimpri Chinchwad College of Engineering, India, mehzabin.pathan@pccoepune.org.

2026. 1049-331X/2026/10-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

A naive version of this comparison is straightforward to conduct but yields misleading conclusions. When evaluated against the files co-edited in the pull request, issue text performs best; when evaluated against the files that define symbols used by the change, structure performs best, and the ranking of methods is nearly inverted. However, neither half of this result survives two controls. The structural half reflects a population effect: a symbol key exists for single-file changes, whereas a co-edited key does not, so the two keys are scored on different pull requests. The lexical half is largely explained by the issue naming the answer: in two tasks in five, the issue names a changed file as code, which a method that matches issue words against file paths can read off directly. Furthermore, a human-annotated key agrees with the co-edited key.

Two findings survive these controls, and this article is organised around them. The first concerns the benchmark more than the methods. Issue-aware retrieval is substantially more valuable on issues that name their files, and a within-task redaction experiment quantifies how much of this value is due to the names themselves. We use the term *leakage* for these names only in this benchmark sense: in deployment, an issue that names `core.py` is a legitimate input, but an average over issues that do and do not name their files describes neither case, and the proportion between them varies across corpora. The second finding is a recommendation: fusing a walk from the file with issue retrieval is robust across keys, strata, unseen repositories, a human-annotated key, alternative cost rules and the loss of the free seed, with the exception of single-file changes, for which the walk alone performs better.

Research questions.

RQ1 Does the uncontrolled comparison between structural and lexical reading orders still hold after the populations are matched and issues that name the answer are controlled for?
RQ2 How much value does issue text contribute to retrieval, and how much of this value stems from the issue naming the files to be changed?
RQ3 Which ordering is robust, and does its performance hold on unseen repositories, on a human-annotated key, under other cost rules, and without a free seed?
RQ4 Beyond this, what does the import graph contribute?

Contributions.

(1) **A protocol for measuring the value of issue text.** An issue-free twin of each ordering captures the total value of the issue, and three nested within-task redactions isolate the portion carried by content that points at the key files. On issues that name a key file as code, the issue contributes +0.156 AUC to the best fusion, of which 0.061 to 0.078 is attributable to the names; on the remaining issues, it contributes +0.016. A dense retriever makes better use of the issue, but the same pattern holds (Section 4.2).
(2) **A robust recommendation, validated out of sample.** Reciprocal-rank fusion of a personalised walk with issue retrieval is never significantly worse than the best pure ordering on the matched population. It held under predictions committed before scoring on seven unseen repositories and on ContextBench's human-annotated key, and it remains the best ordering without a free seed. Whether the walk should favour short files depends on how a context is charged (Section 4.3).
(3) **Controls that altered conclusions in this study**: matching the populations on which two keys are scored; stratifying by whether the issue names a key file as code; and treating the repository as the unit of analysis, which preserves the significance of the headline comparisons but changes the significance verdict for 102 of the 1,076 comparisons that are significant with the pull request as the unit (none of which changes sign).

Table 1. The ground truth against which repository-context systems are evaluated, as reported in the evaluation section of each paper (2026-09-23).

System	Venue	Ground truth used for evaluation
Files touched by the gold patch		
LocAgent [6]	ACL 2025	"The affected files or functions ...as identified in the patches, are considered the target locations"
CoSIL [9]	ASE 2025	File- and function-level accuracy on SWE-bench Lite and Verified, i.e. the patched locations
Agentless [21]	FSE 2025	Correct if the edit covers a superset of the ground-truth patch locations
LingmaAgent [15]	FSE 2025	Fault locations extracted from the developer patch
SWE-Fixer [22]	ACL 2025	Precision and recall against the defective files of the resolving pull request
KGCompass [23]	2025	Files and code elements modified by the ground-truth patch
SWE-Adept [8]	2026	Correct only if all patched locations are in the top-k
SWERank [18]	2025	Every function modified in the pull request
CodeScout [20]	2026	LocAgent's patch-derived targets
RepoGraph [16]	ICLR 2025	Edit locations of the ground-truth patch
Something other than the gold patch alone		
GraphLocator [14]	2025	Patch locations, dual-annotated because patch-derived ground truth "contains missing locations"
SWE-Explore [26]	2026	Code regions that successful agents actually read
ContextBench [12]	2026	Human-annotated context needed to reconstruct the fix
AgentRetrievalBench [17]	2026	Files whose reading helps the agent's next step
CodeR [5]	2024	Golden locations marked by the authors

(4) **Corrections to common arguments in favour of graph retrieval.** The two-hop neighbourhood contains, on average, 87.5% of the needed files and 56% of the repository. Walking against import edges is costly, whereas discarding edge direction is not. Moreover, the advantage that structure shows on a key built from imports is not corroborated by a human-annotated key (Section 4.4).

2 RELATED WORK

Structure and history as guides to change. Impact analysis [4] propagates a change from a modified element along its dependencies; Robillard [19] ranks program elements according to dependency topology; and surveys of feature location [7] observe that structural, textual and dynamic techniques are typically evaluated in isolation. Furthermore, Ko et al. [11] observed that developers navigate largely by names and search. Co-change mining [24, 27] suggests files that tend to change together; our co-edited key adopts this notion as ground truth rather than as a method.

Graph-based context for agents. SWE-bench [10] and its successors [3, 25] established repository-scale change tasks as a standard benchmark, and a family of systems assembles context by traversing a code graph: Aider's repository map [1], RepoGraph [16], LocAgent [6] and CoSIL [9]. In contrast, Agentless [21] relies on file and directory names. Table 1 lists the ground truth against which each system is evaluated, as reported in its own evaluation section. Almost all of them use the files touched by the gold patch (i.e., our co-edited key), and our results support this choice: of our two proxies, it is the one with which a human-annotated key agrees. However, none of them reports results separately for issues that do and do not name the files being scored.

Issues that reveal the answer. SWE-Bench+ [2] finds that 32.67% of the passing patches produced by SWE-Agent with GPT-4 had the solution in the issue or its comments, and Agentless reports issues that contain the exact patch. A weaker form of this leakage concerns only *location*: a retrieval

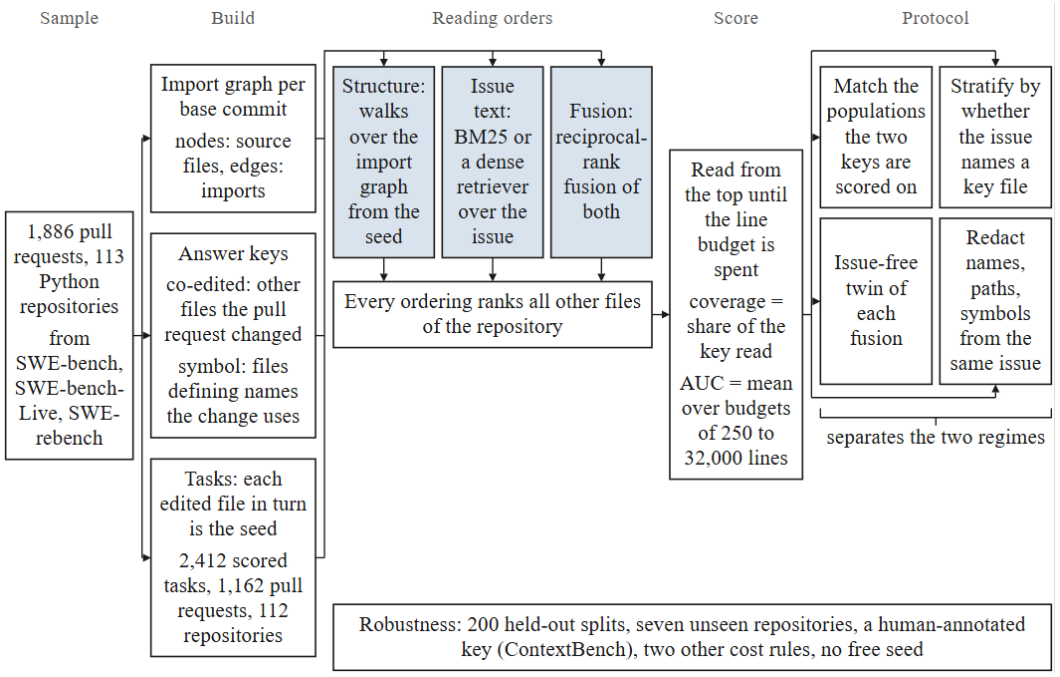


Fig. 1. Overview of the study: the sample, what is built from each pull request, the three kinds of reading order, the metric, and the protocol that separates issues that name a key file from those that do not.

step needs only to know which files to open. This form is also known. Agentless classifies the location information in SWE-bench Lite and finds that around half of its issues already provide the location of the file to be edited. Similarly, the SWE-Bench Illusion study [13] shows that models identify the buggy file from the issue alone in up to 76% of SWE-bench Verified tasks, compared with 53% on repositories outside SWE-bench. Our rate, 40.2% of tasks naming a key file as code, is consistent with the first of these findings. Our contribution is not the rate itself but its consequence: how much of an issue-aware method's score it accounts for, separated within each task from the rest of the issue's content.

Concurrent work on the key. SWE-Explore [26] derives ground truth from the code that successful agents read and scores ranked lists under a line budget, which is close to our cost model; ContextBench [12] annotates gold context manually; AgentRetrievalBench [17] scores files according to whether they help the next step; and GraphLocator [14] dual-annotates the patch-derived key. We use ContextBench's key as a third, independent answer key (Section 4.3).

3 STUDY DESIGN

Figure 1 gives an overview of the study. The corpus, graph builder, task definition and metric were fixed and written down before any method was run (the *design freeze*). The method families, including the fusion among them, were added afterwards in response to the first results; the fusion therefore originated as an exploratory result. We subsequently tested it twice under predictions written down before any of the data they concern were scored (`paper/external_predictions.md`, `paper/contextbench_predictions.md`). Because neither record was deposited with a third party, we do not describe either step as a pre-registration. Analyses that were not fixed in advance (the

cost rules, the dense retriever, the systems' selectors and the condition without a free seed) are marked as exploratory where they appear.

The pipeline's code, including sampling, graph construction, statistics, tables and figures, was written with an AI coding assistant (Claude Code, Anthropic) under the authors' direction. The assistant also ran the pipeline, helped debug it, and proposed detailed design choices, among them the graph builder's rules, the sampling parameters and the statistical tests. Each choice is recorded with its reason in the replication package and was accepted by the authors, who are responsible for every choice and claim. The code is covered by unit tests, and a checker recomputes every number quoted in the text from the result files.

3.1 Corpus, tasks and keys

We drew 1,886 pull requests from SWE-bench, SWE-bench-Live and SWE-rebench in Python repositories that satisfy our sampling rules, with at most 20 per repository. A pull request that edits k existing non-test source files that are nodes of the import graph yields k tasks: each edited file in turn serves as the *seed*, and the task is to order the remaining files of the repository. This procedure yields 3,123 tasks from 1,873 pull requests in 113 repositories (13 pull requests edit no graph file). Of these, 2,412 tasks (1,162 pull requests in 112 repositories) have at least one key file once the seed is removed, and only these tasks are scored. Of the 711 unscored tasks, 706 originate from pull requests that change a single source file; they account for 61.2% of all 1,154 such tasks, because the symbol key described below is empty whenever the changed code resolves to nothing outside itself. In the median task, the repository has 131 graph nodes and the key contains 2 files.

Graphs are constructed for each base commit using an `ast` walker: nodes are source files, an edge $a \rightarrow b$ indicates that a imports a module resolving to b, test files are excluded, and the per-file line counts obtained in the same pass serve as the cost unit. We define two keys, which are never merged:

`co_edited` The other non-test source files changed by the same pull request, representing what a developer actually touched, independently of the import graph. This key is available only when a pull request edits more than one graph file: 1,958 tasks from 708 pull requests.

`symbol` Files that define names used on the lines added by the pull request, resolved with `jedi`'s `goto` using `follow_imports=True` on the patched code, representing what the change depends on. This key covers 2,341 tasks from 1,156 pull requests.

Because the symbol key is constructed by following imports, it is partly circular with respect to the structural orderings, and we therefore treat it as the secondary proxy. The 702 pull requests that carry both keys constitute the *matched population*, on which every cross-key comparison is repeated. Of the 1,156 pull requests with a symbol key, 454 have no co-edited key because they edit a single graph file; we refer to these as *single-file* pull requests. By *issue text*, we refer to the problem statement only, excluding the maintainers' `hints_text`.

A third key, used only in Section 4.3, is ContextBench's human-annotated gold context [12]: the Python files that an annotator judged necessary to reconstruct the fix, excluding the seed. We processed its SWE-bench Verified and SWE-bench Pro Python tasks through the unchanged pipeline, sampled according to our sampling rules: 108 pull requests in 14 repositories have a non-empty gold key, 6 of which are also in the main sample.

3.2 Reading orders

Every ordering receives the graph, the seed, per-file identifier bags and the issue text, and ranks every other file (Table 2). Identifier bags are obtained by splitting identifiers on underscores and camel case, and the issue is tokenised in the same way. BM25 uses $k_1 = 1.5$, $b = 0.75$ and differs

Table 2. Reading orders. Structural orderings use the seed and the graph; global orderings use neither the seed nor the issue; lexical orderings use no graph.

Family	Method	Ordering
structure	`bfs_und`, `hops_lines`	breadth-first from the seed; by (hops, file length)
	`ppr_und`, `ppr_und_pl`	personalised PageRank from the seed, undirected
	`ppr_out`, `ppr_out_pl`	the same walk along import edges: what the seed uses
	`ppr_in`, `ppr_in_pl`	the same walk against them: what uses the seed
global	`pagerank`, `indegree`	repository prior; ignores seed and issue
lexical	`bm25_seed`, `path_seed`	BM25 over contents or paths, queried by the seed
	`bm25_issue`, `bm25_issue_pl`	BM25 over contents, queried by the issue
	`path_issue`	BM25 over file paths, queried by the issue
fusion	`rrf_hops_path`	RRF of `hops_lines` and `path_issue`
	`rrf_hops_issue_path`	RRF of `hops_lines`, `bm25_issue`, `path_issue`
	`rrf_pprpl_issue_path`	RRF of `ppr_und_pl`, `bm25_issue`, `path_issue`
	`rrf_ppr_issue`, `rrf_ppr_issue_path`	the same on the unscaled `ppr_und`
reference	`same_dir`, `random`, `oracle`	seed's directory first; shuffled; key files only, cheapest first

across methods only in the query. Fusion is performed with reciprocal-rank fusion (RRF) with $k = 60$. Variants with the suffix `_pl` divide a score by file length. Four orderings recur throughout this article, and in the text we refer to them by plain names: *the fusion* (`rrf_pprpl_issue_path`), *the length-aware walk* (`ppr_und_pl`), *the hop walk* (`hops_lines`) and *path matching* (`path_issue`). Note that structural orderings are given a correct seed, whereas issue orderings receive no seed at all.

Three further kinds of ordering were added after the first results; they are scored but excluded from the ranks and correction families of the other tables, so that their addition does not change any result reported elsewhere. First, a *dense retriever*, `jina-embeddings-v2-base-code` (the model our design fixed before any method was run), ranks files by the cosine similarity between their first 512 tokens and the issue (`dense_issue`), which is embedded as a whole as the length-weighted mean of its 512-token chunks. Its fusions replace or complement BM25 over contents. Second, the *issue-free twins* of the fusions replace every issue-derived input with its seed-derived counterpart. Third, the *context selectors of three published systems* (Aider's repository map, RepoGraph's two-hop ego graph and LocAgent's typed breadth-first search, reimplemented from their papers) are scored as orderings, with the files they do not select appended in random order.

3.3 Metric

An ordering is read from the top, one whole file at a time, until the next file would exceed the line budget. Coverage is defined as the fraction of the key that has been read. We evaluate at budgets ranging from 250 to 32,000 lines in doublings and report the AUC, i.e., the mean coverage over the eight budgets; ties are re-drawn three times and the results are averaged. Python source averages 8.50 tokens per line in aggregate over the 121,075 distinct file versions (`cl100k_base`), so a budget of 8,000 lines corresponds to roughly 68,000 tokens. Restricting the AUC to budgets of up to 8,000 lines does not change the leading orderings.

Because AUC has no intuitive scale, we anchor it as follows. On the matched population, a random ordering scores 0.249 and the oracle 0.774 on the co-edited key (0.252 and 0.779 on the symbol key), so a difference of 0.05 AUC corresponds to about a tenth of the distance from random to oracle; the recommended fusion covers 45% of this distance on the co-edited key and 41% on the symbol key. We also report the number of lines read before half of the key is covered (Table 17 in the appendix).

The rule that reading stops at the first file that overflows the budget is part of the frozen metric. Because this rule favours short files, Section 4.3 additionally scores every ordering under a rule that skips such a file and continues reading, and under a rule that charges each file one unit regardless of its length.

3.4 Statistics

Task AUCs are first averaged within each pull request. Paired comparisons use the Wilcoxon signed-rank test with the repository as the unit of analysis (i.e., pull-request scores averaged within each of the 112 repositories), because pull requests are clustered in repositories that differ in size, layout and issue-writing culture. Effect sizes are reported as Vargha–Delaney A_{12}, and confidence intervals are obtained by bootstrapping over repositories. Holm correction is applied within families fixed by the design (Table 20): each combination of key, unit, population and leakage stratum constitutes one family of 325 tests among twenty-six orderings. Ranks are computed among the twenty-two ranked orderings, excluding the oracle, so that the best ordering is ranked first. Differences quoted in the text are taken from the repository-level Wilcoxon analysis; they are means of per-repository differences and can therefore differ in the third decimal from the pull-request means reported in the tables. The mixed-model estimates of the same comparisons are reported in Table 21 and can differ slightly from them.

The choice of unit matters for borderline comparisons: of the 1,076 comparisons that are significant after Holm correction with the pull request as the unit, 102 are not significant with the repository as the unit, and the reverse holds for 2. No estimate changes sign. In addition, every comparison on which a claim rests is re-estimated as a linear mixed model of the per-pull-request difference with a random intercept per repository (Table 21); for 40 of these 41 comparisons, the mixed model reaches the same verdict. The exception is the comparison between the hop walk and path matching on co-edited files, for which the mixed model finds path matching better by 0.044 ($p_{\text{Holm}} < 10^{-3}$), whereas the repository-level test does not; no claim depends on this comparison being null.

4 RESULTS

4.1 RQ1: the naive comparison and why it does not hold under controls

Table 3 presents the comparison as it is commonly conducted: each key over its full population, without any control. Figure 2 plots coverage against the budget for three of these orderings. Under the co-edited key, the five best orderings comprise four fusions and one lexical method, and the best purely structural ordering ranks ninth. Under the symbol key, the four best orderings comprise three structural orderings and one fusion, and the best lexical ordering ranks fourteenth. The individual reversals are large and significant: path matching outperforms the length-aware walk by 0.059 AUC on co-edited files ($p_{\text{Holm}} < 10^{-4}$) but underperforms it by 0.100 on symbol files ($p_{\text{Holm}} < 10^{-7}$). Taken at face value, these results indicate that the choice of key determines which signal performs best.

The structural half of this reversal reflects a population effect. The two columns are scored on different pull requests: 454 of the pull requests in the symbol column are single-file changes, which have no co-edited key and are precisely the cases in which a seed's import neighbourhood contains most of what there is to find. On the 702 pull requests that carry both keys (Table 7), we observe that nothing changes on the co-edited key, whereas on the symbol key a walk along import edges falls from first to seventh (0.491 to 0.417) and path matching rises from fourteenth to eighth. The same comparison that gave the walk a 0.101 lead on the full corpus ($p_{\text{Holm}} < 10^{-7}$) yields +0.008 and $p_{\text{Holm}} = 1$ on the matched population; for the hop walk against path matching, it yields +0.019

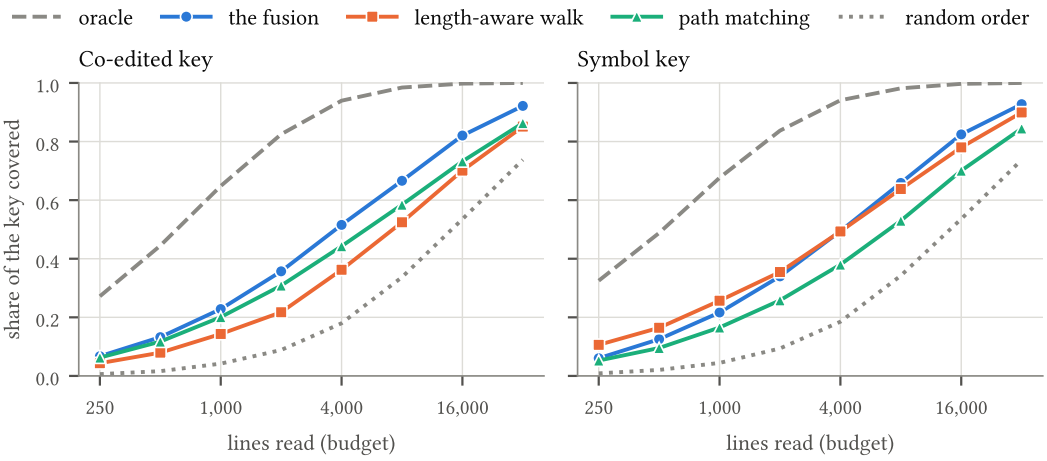


Fig. 2. Mean coverage of the answer key against the number of lines read, over the scored tasks of each key and without either control, for three orderings and the two references. The AUC is the mean coverage over the eight budgets shown.

Table 3. The uncontrolled comparison: mean AUC by key, with the pull request as the unit and bootstrap 95% confidence intervals over repositories. Neither control is applied; the text shows that both halves of the reversal disappear once they are applied.

co_edited (708 PRs)				symbol (1,156 PRs)			
#	Method	Fam.	AUC [CI]	#	Method	Fam.	AUC [CI]
–	oracle	ref	0.773 [.750,.796]	–	oracle	ref	0.793 [.772,.813]
1	rrf_pprpl_issue_path	fus	0.486 [.455,.517]	1	ppr_out_pl	str	0.491 [.463,.517]
2	rrf_hops_issue_path	fus	0.480 [.448,.510]	2	ppr_und_pl	str	0.489 [.460,.517]
3	rrf_hops_path	fus	0.457 [.425,.489]	3	hops_lines	str	0.477 [.447,.505]
4	rrf_ppr_issue_path	fus	0.445 [.414,.475]	4	rrf_hops_path	fus	0.476 [.449,.502]
5	path_issue	lex	0.434 [.405,.465]	5	rrf_pprpl_issue_path	fus	0.474 [.445,.501]
6	bm25_issue	lex	0.407 [.379,.435]	6	ppr_out	str	0.458 [.430,.487]
7	bm25_seed	lex	0.393 [.362,.425]	7	rrf_hops_issue_path	fus	0.457 [.429,.483]
8	rrf_ppr_issue	fus	0.393 [.363,.425]	8	ppr_und	str	0.446 [.417,.475]
9	hops_lines	str	0.389 [.354,.423]	9	bfs_und	str	0.420 [.391,.447]
10	ppr_und_pl	str	0.371 [.338,.403]	10	rrf_ppr_issue_path	fus	0.420 [.392,.448]
11	path_seed	lex	0.371 [.338,.404]	11	pagerank	glo	0.410 [.381,.439]
12	bfs_und	str	0.362 [.330,.395]	12	indegree	glo	0.398 [.371,.425]
13	ppr_und	str	0.357 [.325,.391]	13	rrf_ppr_issue	fus	0.390 [.361,.421]
14	bm25_issue_pl	lex	0.354 [.318,.389]	14	path_issue	lex	0.382 [.352,.412]
15	same_dir	ref	0.329 [.298,.360]	15	path_seed	lex	0.365 [.336,.392]
16	ppr_out_pl	str	0.327 [.296,.358]	16	bm25_issue_pl	lex	0.355 [.324,.386]
17	ppr_out	str	0.312 [.282,.344]	17	bm25_issue	lex	0.340 [.311,.368]
18	ppr_in_pl	str	0.310 [.279,.341]	18	bm25_seed	lex	0.336 [.305,.365]
19	ppr_in	str	0.308 [.278,.339]	19	same_dir	ref	0.320 [.290,.349]
20	pagerank	glo	0.271 [.240,.301]	20	ppr_in_pl	str	0.277 [.248,.303]
21	indegree	glo	0.271 [.240,.301]	21	ppr_in	str	0.269 [.242,.295]
22	random	ref	0.248 [.220,.277]	22	random	ref	0.248 [.220,.276]

and $p_{\text{Holm}} = 1$. Consequently, the top three places under both keys are held by the same three fusions.

The lexical half is largely explained by the issue naming the answer. The full path of a key file appears verbatim in the issue for 12.6% of tasks, and its bare stem for 48.5%. A stem written as code (`core.py`, `simbad/core`, a dotted module path, inside backticks or within a code block) is referred to as an *explicit* mention, and a stem written only as an ordinary word as an *incidental* mention. Explicit mentions occur in 40.2% of tasks. Removing them, together with applying the population control (Table 8), moves path matching on co-edited files from fifth to twelfth and pure structure from ninth to fifth. In addition, the advantage of the hop walk over path matching changes from −0.007 under the full-path control ($p_{\text{Holm}} = 1$) to +0.042 under the primary control ($p_{\text{Holm}} = 0.60$), on 531 pull requests. As a result, the ordering that appeared to be the best single signal on this key becomes indistinguishable from one that never reads the issue. The issue does not name the seed file more often than the key files, so the seed does not explain this effect: its full path appears in 10.3% of tasks.

Which proxy to trust. In cases where the two keys still disagree, a human-annotated key provides the deciding evidence. On ContextBench's tasks, it agrees with the co-edited key: the ranking under the human key agrees with that of the co-edited key at Kendall $\tau = 0.97$ and with that of the symbol key at 0.59. Moreover, the length-aware walks that rank first and fourth under the symbol key rank tenth and fourteenth under the human key (Table 13). Therefore, the answer to RQ1 is negative: once the populations are matched and the naming is controlled, neither signal family dominates, and the key that the field already uses is the better of the two proxies. The remainder of this section examines how much the issue is then worth and which ordering is safe to use.

4.2 RQ2: the value of the issue and the contribution of the names

Stratifying by whether the issue names a key file compares two kinds of issue in addition to two degrees of leakage: a report that contains a stack trace is plausibly longer and more tractable than a one-line feature request. We therefore measure the value of the issue twice, once across tasks and once within tasks.

Across tasks: an issue-free twin. Each fusion is paired with a twin in which every issue-derived input is replaced by its seed-derived counterpart; for example, `rrf_pprpl_issue_path` is paired with `rrf_pprpl_seedpath`, which fuses the same walk with the same two BM25 rankings queried by the seed's own path and contents. The twin combines the same number of lists but never reads the issue, so the difference between the two measures the value of the issue, net of the effect that "a fusion simply receives more input". On the matched population, the issue contributes +0.069 AUC to the fusion on the co-edited key and +0.055 on the symbol key. On tasks whose issue names a key file as code, these values increase to +0.156 and +0.121; on the remaining tasks, they are +0.016 and +0.012, and neither survives correction.

Within task: redaction. We then take the *same* issue and delete the content that points at the key files, while preserving the prose, the traceback and the length. Because what counts as "pointing" is a design choice, we define it in three ways. Specifically, the *names* arm deletes only explicit mentions of the key files' stems; the *paths* arm additionally deletes every component of their paths; and the *symbols* arm additionally deletes every function, class and method name that they define, in each case only where written as code. The names arm provides a lower bound on leakage, whereas the wider arms over-remove content, since a directory called `models` is also ordinary vocabulary; the three arms therefore bracket the effect. Of the 970 tasks whose issues name a key file, the 848 in the matched population (378 pull requests in 105 repositories) are scored. Removing only the

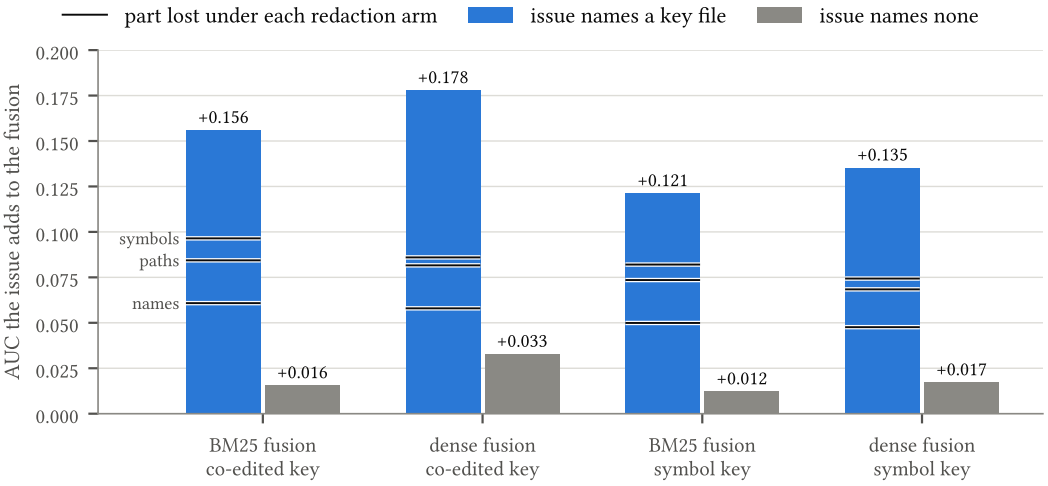


Fig. 3. What the issue adds to each fusion, measured as the fusion minus its issue-free twin, on tasks whose issue names a key file as code and on the remaining tasks. The lines across a bar mark how much of that value is lost under the names, paths and symbols redaction arms. Matched population, repository as the unit; the values are those of Table 4.

names reduces the AUC of path matching by 0.110 on co-edited files and that of the fusion by 0.061; the wider arms reduce the fusion's AUC by 0.084 and 0.096 (Table 4, Figure 3). On the issues that name nothing, the names arm has no effect, as expected by construction, whereas the wider arms reduce AUC by 0.016 to 0.018, which corresponds to the part they over-remove.

The decomposition. Because both experiments run on the same population with the same unit, their results are additive. On issues that name a key file, the issue contributes +0.156 to the fusion on co-edited files; on the remaining issues, it contributes +0.016. Of this +0.140 gap, the names arm attributes 0.061 to the names, and the widest arm, net of what it also removes where nothing is named, attributes 0.078 (on the symbol key, 0.050 to 0.065 of a +0.109 gap). Hence, between two fifths and a half of the difference between the two kinds of issue is due to the naming itself. The remainder reflects a difference between the issues, or between the tasks, that none of the redactions we could define captures: even path matching, which reads only file paths, scores 0.393 on named-file tasks after the widest redaction, compared with 0.333 on the other tasks.

A better reader. All of the above analyses read the issue with BM25, so "what the issue is worth" could characterise the retriever rather than the issue. The dense retriever makes better use of the issue: on its own, it exceeds BM25 over contents by 0.059 and 0.054, and when substituted for BM25 in the fusion, it adds 0.018 on both keys (Table 10). On issues that name nothing, the dense fusion gains 0.033 from the issue on co-edited files, twice the gain of the BM25 fusion and, unlike it, statistically significant; on the symbol key, it gains 0.017, which is not significant. Part of the small value of unnamed issues was therefore attributable to BM25. However, this gain is still a fifth of what the same fusion gains where the issue names a file (0.178), and the names account for a similar share: of the 0.145 stratum gap, 40 to 47 per cent, compared with 43 to 56 per cent for BM25. We did not evaluate a reader that processes the issue as a whole, such as an LLM reranker; the redaction protocol applies to such a reader without modification.

Table 4. The value of the issue to the BM25 fusion (`rrf_pprpl_issue_path`) and the dense fusion (`rrf_pprpl_dense_path`), and the cost of each redaction arm, on tasks whose issue names a key file as code and on the remaining tasks. Worth is the fusion minus its issue-free twin. Matched population, repository as the unit; * survives Holm correction.

	co_edited		symbol	
	BM25	dense	BM25	dense
issue worth, named	+0.156*	+0.178*	+0.121*	+0.135*
issue worth, not named	+0.016	+0.033*	+0.012	+0.017
names arm, named	+0.061*	+0.058*	+0.050*	+0.048*
paths arm, named	+0.084*	+0.082*	+0.074*	+0.068*
symbols arm, named	+0.096*	+0.086*	+0.082*	+0.074*
symbols arm, not named	+0.018*	+0.018*	+0.017*	+0.017*

What these results imply. They do not imply that issue text is useless: an ordering that exploits a real report naming a real file exploits information that genuinely exists. Rather, they show that a benchmark assembled from issue reports measures two regimes simultaneously, in proportions that vary across corpora, and that these regimes can be separated at the cost of a few regular expressions and one additional run. Consequently, any evaluation on SWE-bench-derived data that reports a gain from issue-aware retrieval without the stratification and the redaction reports the sum of the two.

4.3 RQ3: a robust ordering and its generalisability

If the answer key is not known in advance (and for a tool shipped to users, it is not), the quantity to optimise is the worst-case performance of a method across keys, measured as its shortfall behind the best ordering under either key. On the matched population, the best ordering by this criterion is the fusion, which is the best ordering under both keys; it exceeds the best pure lexical ordering by 0.055 and 0.067 and the best pure structural ordering by 0.097 and 0.039. It is never significantly worse than the best pure ordering in any leakage stratum on the matched population. On the full symbol-key population, however, it is: once issues that name a key file are removed, the length-aware walks outperform it by 0.049 to 0.062 ($p_{\text{Holm}} < 10^{-4}$), because that population retains the single-file pull requests. On these pull requests, the five best orderings are all structural, and for a change that touches one file, the walk alone is the better choice.

Because the fusion was identified after the results had been observed, we tested it further. Table 5 reports every test, and Table 6 lists the eight predictions committed before scoring.

Held-out repositories. Across 200 random 56/56 splits of the repositories, a mechanical selection criterion applied to one half selects the fusion in 198 splits. In 195 of these, it is the best ordering on the co-edited key of the other half, and in 198 of 198 on the symbol key (Table 14). This result demonstrates that the choice is stable across our repositories, but it does not demonstrate that the choice generalises.

Seven unseen repositories. Seven repositories satisfy all of our sampling rules and were excluded only because their tasks originate from SWE-Gym and SWE-bench Pro. We drew 140 pull requests from them according to the same rules and committed four predictions before building anything. Three of these predictions hold: the fusion ranks second under both keys, behind only another fusion built on the hop walk, with a worst-case shortfall of 0.030 (exactly at the limit we set rather than within it); it is never significantly worse than a pure ordering; and walking inward is as costly as in the main sample. The fourth prediction fails: deleting the names reduces the fusion's AUC by

Table 5. The recommended fusion under every check: its rank (oracle excluded) and its AUC shortfall relative to the best ordering, per key. Rows marked † were tested against predictions committed before scoring; the remaining rows are exploratory.

		co_edited (or human key)		symbol	
Check	Ordering	rank	shortfall	rank	shortfall
Main sample, matched population	`rrf_pprpl_issue_path`	1	.000	1	.000
Main sample, full population	`rrf_pprpl_issue_path`	1	.000	5	.017
Held-out halves, 200 splits	`rrf_pprpl_issue_path`	best in 195/198	.002	best in 198/198	.000
Skip an overflowing file	`rrf_pprpl_issue_path`	2	.002	1	.000
Charge per file	`rrf_pprpl_issue_path`	6	.055	4	.045
† Seven unseen repositories	`rrf_pprpl_issue_path`	2	.030	2	.008
† Human key (ContextBench)	`rrf_pprpl_issue_path`	2	.009	–	–
No free seed (dense seed)	`rrf_pprpl_dense_path`	1	.000	1	.000

Table 6. The predictions committed before the external repositories (E) and the ContextBench tasks (G) were scored, and the corresponding outcomes. Mixed model with a random intercept per repository, Holm correction within each prediction.

	Prediction	Observed	Outcome
E1	fusion in the top three non-oracle orderings under both keys, worst shortfall ≤ 0.030	rank 2 and 2; shortfall 0.030	pass
E2	fusion never significantly worse than four pure orderings, either key	smallest difference +0.058 ($p_{\text{Holm}} = 0.511$)	pass
E3	outward walk beats inward on the symbol key	+0.169 ($p = 0.001$)	pass
E4	names redaction costs the fusion under both keys; nothing where nothing is named	+0.030 ($p_{\text{Holm}} = 0.112$) and +0.033 ($p_{\text{Holm}} = 0.008$); control ≤ 0.003	**fail**
G1	fusion in the top three non-oracle orderings on the human key, shortfall ≤ 0.030	rank 2; shortfall 0.009	pass
G2	fusion never significantly worse than four pure orderings	smallest difference +0.086 ($p_{\text{Holm}} < 0.001$)	pass
G3	outward walk beats inward	+0.083 ($p = 0.006$)	pass
G4	Kendall $\tau \geq 0.5$ between the human key's ranking and each proxy's	symbol 0.59, co-edited 0.97	pass

0.030 on the co-edited key with $p_{\text{Holm}} = 0.11$, on only 33 matched pull requests in six repositories, so the causal effect of the names alone is not established for this set. The wider arms reduce it by 0.086 and 0.090: in these larger repositories, issues refer to files through their directories more than through their names.

A human-annotated key. On ContextBench's tasks, all four predictions hold. The fusion is the second-best ordering on the human key, 0.009 behind a fusion of the same family, and it achieves a higher AUC than each of four named pure orderings, by 0.086 to 0.128 ($p_{\text{Holm}} \leq 0.008$). Its annotators worked from the fix, and 108 pull requests constitute a small set; nevertheless, it is the only key in this article that was not constructed by any part of our pipeline.

Other cost rules. Under the frozen rule, preferring short files approximates a knapsack heuristic for the metric, so the length-aware walk could owe its position to the rule itself. Under a rule that skips a file that does not fit and continues reading, the fusion is the best ordering on the

symbol key and 0.002 behind the fusion built on the hop walk on the co-edited key. However, the margin gained from length-awareness shrinks by about sixty per cent (the two fusions that differ only in this respect differ by +0.015 and +0.021 instead of +0.042 and +0.050). Under a rule that charges each file one unit, length-awareness becomes a disadvantage, and the unscaled fusion `rrf_ppr_issue_path` is the best ordering under both keys. In summary, a fusion is the best ordering under every rule; whether its walk should favour short files depends on whether a context is charged by size, as a token window is, or by the number of files opened (Table 11).

Without a free seed. In practice, an agent is not given the file to change. For each pull request, we therefore let the issue select the seed (the top-ranked file of the dense retriever), walk from that guess and fuse, and we score the result against every edited file (including the true seeds, so that identifying the file is part of the task) and against the symbol key. The guess is the correct file in only 34% of pull requests; nevertheless, walking from it and fusing with the dense reader yields the best ordering under both keys, at 0.516 and 0.475 (Table 12). It outperforms the dense retriever alone by 0.023 on the edited files ($p_{\text{Holm}} < 10^{-3}$) and 0.058 on the symbol key, and ties with it where the issue names no edited file (−0.001). Seeding the walk with a BM25 guess instead, which is correct in 41% of pull requests, performs worse, which indicates that the reader that selects the seed should be the same reader that reads the issue in the fusion.

Published systems' selectors. When scored as orderings on the matched population, the context selectors of Aider, RepoGraph and LocAgent rank between tenth and twentieth. The recommended fusion exceeds each of them by 0.115 to 0.176 on co-edited files and by 0.084 to 0.155 on symbol files ($p_{\text{Holm}} < 10^{-4}$). Aider's repository map and RepoGraph's two-hop ego graph also rank below the plain length-aware walk, while LocAgent's typed search ties with it on co-edited files. Note that these are the selectors alone, reimplemented from the original papers without the language model that each system wraps around them; the comparison therefore reflects what their graph component contributes as a reading order, not how well the systems perform.

The fusion's own parameter. The RRF parameter k was fixed at 60 without prior testing; at $k = 10$ and $k = 200$, the fusion's AUC changes by at most 0.007 on the co-edited key and 0.010 on the symbol key.

4.4 RQ4: the contribution of the import graph

The two-hop neighbourhood covers half of the repository. Specifically, 86.7% of symbol-key files and 88.1% of co-edited files lie within two undirected hops of the seed (Table 15, Figure 4), which at first sight appears to be strong evidence in favour of graph retrieval. However, the median two-hop ball contains 68 files and about 21,600 lines, more than five times a 4,000-line budget. Relative to its own repository, it covers a median of 58.5% (a mean of 56%), with a median precision of 3.4%, and it improves recall over a random sample of the same size by a factor of only 1.57. The two-hop neighbourhood is therefore not an effective filter: it indicates only that the needed files lie somewhere within half of the repository, which is precisely where the *ordering* determines the outcome.

Direction: reversing a walk is costly, whereas discarding direction is free. Walking along import edges, towards what the seed depends on, places `ppr_out_pl` first on the symbol key at 0.491, whereas walking against them drops to 0.277 (twentieth), a gap of 0.215 on the full population ($p_{\text{Holm}} < 10^{-14}$). The undirected walk loses almost nothing (0.489). On the co-edited key, every directed walk ranks below `same_dir`, which consults no graph at all. Because the symbol key is constructed by resolving names along imports in one direction, a walk in that direction measures something close to the key's own construction.

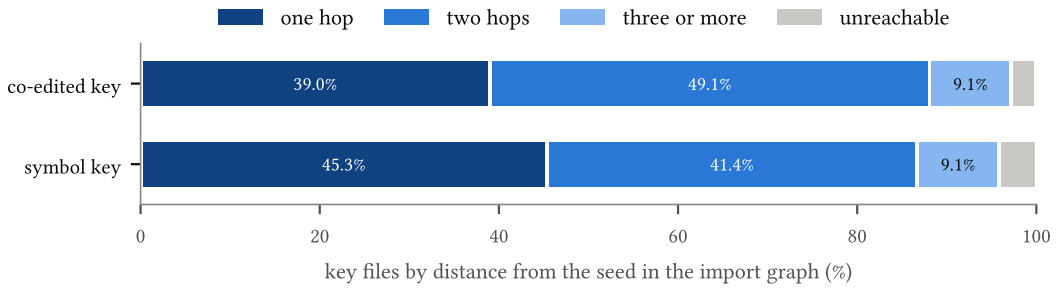


Fig. 4. Key files by undirected hop distance from the seed in the import graph, per key (2,412 tasks; the values are those of Table 15).

The contribution of a repository prior. A query-independent PageRank, which never considers the seed or the issue, reaches 0.410 on symbol files compared with 0.491 for the best personalised walk (67% of the walk's margin over random), but only 0.271 on co-edited files, barely above random (0.248), where the walk exceeds it by 0.101. This indicates that knowing which file is being changed matters considerably for predicting what else was changed, and much less for predicting what the code calls.

The signals retrieve different files. At $k = 20$, 24.4% of co-edited files are found only by the lexical family and 9.5% only by the structural family; on symbol files, the asymmetry is reversed (20.0% against 16.5%, Table 16). This complementarity explains why fusing the two families is effective; where the issue names a key file, much of the lexical family's share is attributable to that naming (Section 4.2).

What the human key reveals about the graph. On the gold files that a fix reads but does not edit (the context that a graph walk is supposed to be best at finding), walks and issue-only retrievers are indistinguishable on the 85 ContextBench pull requests that contain such files (differences of 0.006 to 0.012, $p_{\text{Holm}} = 1$), and the fusion outperforms the issue-only orderings by 0.082 and 0.084 ($p_{\text{Holm}} \leq 0.008$). Note that this analysis was added after the predictions were scored.

The budget is rarely the binding constraint. An oracle fits the complete key within 8,000 lines for 97.5% of pull requests, whereas the best ordering completes 58.5% on the co-edited key and 51.0% on the symbol key (Table 18). Because the oracle reads only key files, the forty-point gap is an upper bound on what a better ordering could add rather than an estimate; what it establishes is that an ordering that fails to cover the key at this budget fails because of its order.

5 DISCUSSION

Implications for benchmarks. The field's default key, namely the files touched by the gold patch, is the better of our two proxies: a human-annotated key agrees with it rather than with the key built from imports. The field's omission lies elsewhere. In this corpus, two tasks in five have an issue that names a key file as code, and the score of an issue-aware method mixes that regime with the other. The control requires only a regular expression over text that the benchmark already provides, and the redaction requires one additional run. We believe that retrieval results on issue-derived benchmarks should report the naming rate, the stratified result and the redaction as routinely as the corpus size.

Implications for tools. Tool builders should fuse a walk from the file being changed with issue retrieval, read the issue as effectively as possible (in our study, a dense retriever outperforms BM25), and make the walk favour short files only if the context is charged by size. For a change confined to one file, the walk alone is the better choice. When the file to change is not known, the same reader should select the starting point. The two-hop neighbourhood is not a useful filter on its own, and direction matters only insofar as walking against imports is costly.

What the baselines do not show. Our orderings are deliberately simple (BM25, PageRank and one embedding model), and the systems' selectors are reimplemented from their papers rather than run end to end. A skeptical reader could interpret the fusion result as two weak but complementary signals outperforming each signal alone. We acknowledge that this interpretation is accurate; the value of the fusion lies in its robustness, its low cost and the fact that it survives every test we could devise. The methodological contribution does not depend on it: the stratification and redaction protocol applies to any reader of the issue, including an LLM reranker, which we did not evaluate.

Scope and remaining work. Every experiment in this article runs on a CPU without paid model access, and within this budget we extended the evaluation as far as possible: two proxy keys, a human key, unseen repositories, a dense retriever, three systems' selectors, and a condition without the free seed. What these experiments cannot show is whether better retrieval increases the rate at which an agent resolves the issue. Answering this question requires model calls at scale, and Appendix C specifies the experiment in advance so that it can be run without modification. Its analysis would inherit the protocol introduced in this article (stratifying by naming, the issue-free twin, the redaction and the repository as the unit), which constitutes the contribution we offer here.

Recommendations. (1) Report retrieval results separately for each answer key, and prefer the patch key over an import-derived key. (2) Report, and stratify by, whether the query names the answer files as code, and apply the redaction to any method that reads the issue. (3) Compare each issue-aware method against the same method with a seed-derived query. (4) Match the populations before comparing results across keys. (5) Cluster statistical tests by repository. (6) State explicitly how reading is charged.

6 THREATS TO VALIDITY

Construct validity. Neither proxy key constitutes ground truth about what a developer needed to read. The co-edited key conflates files changed with files read; the symbol key is circular with respect to the structural orderings; and the human key covers 108 pull requests and records what the fix required as reconstructed by annotators who had seen the fix. Coverage assumes that whole files are read; the alternative cost rules indicate which conclusions depend on this assumption.

Internal validity. The graph builder does not capture dynamic imports or plugin registration. The symbol key inherits `jedi`'s resolution failures, which depend on the environment: rebuilding the keys of all 1,886 sampled pull requests on a second machine left every co-edited key identical but changed the symbol key of 41 pull requests, typically by one file. Re-scoring changes no AUC by more than 0.003 and alters no conclusion. In the held-out splits of Section 4.3, the fusion is then selected in 194 of 200 splits instead of 198, and confirmed on the co-edited key in 191 of these instead of 195. This article reports the keys as originally built. The leakage classifier is a regular expression and will mislabel some mentions, which is why we report three nested strata and three redaction arms. Leakage that we did not anticipate redacting, such as a paraphrase of a function's behaviour, would remain in the residual; the leakage share is therefore bounded more firmly from below than from above.

External validity. Our study is limited to Python and to file granularity, and its pull requests come from benchmarks that select for reproducible, test-verified fixes, which may be better documented than average and may therefore overstate the effectiveness of issue-text methods. Very large repositories are under-represented; nevertheless, the pattern in Table 3 is stable across size quartiles (Table 19). Finally, the external set comprises seven repositories, and one of its four predictions failed.

Conclusion validity. The fusion family was selected after the results had been observed. The held-out splits, the external repositories and the human key test it, but none of them can undo the fact that we selected it, from five candidates, on these data.

7 CONCLUSION

What belongs in an agent's context window, given the file it is changing? When compared naively, import-graph proximity and issue-text matching exchange positions depending on the answer key. This reversal is a property of the comparison rather than of the signals: one half of it stems from the different pull requests that the two keys cover, and the other from the issues that name their files. Moreover, a human-annotated key agrees with the key that the field already uses. Two contributions survive: a protocol and a recommendation. Issue-aware retrieval is worth +0.156 AUC where the issue names a changed file, compared with a non-significant +0.016 elsewhere, and about half of this difference is due to the naming itself; benchmarks should therefore report the two regimes separately. Furthermore, fusing a walk from the file with issue retrieval is robust across keys, unseen repositories, a human key, cost rules and the loss of the free seed, with the exception of single-file changes, for which the walk alone performs better.

DATA AVAILABILITY

The replication package is available at https://github.com/Neil-Lunavat/tosem-dependency-graph. It contains the graphs, tasks, keys, issue bags, orderings and scores; the external and ContextBench trees together with the predictions committed before they were scored; the scripts that compute the dense embeddings, which are regenerated rather than distributed; and every analysis script. A checker recomputes every number quoted in the text from the result files and fails if any sentence disagrees with them. The package will be archived under a permanent identifier upon acceptance.

REFERENCES

[1] Aider. 2023. Building a Better Repository Map with Tree Sitter. https://aider.chat/2023/10/22/repomap.html. Blog post, 22 October 2023. Accessed 2026-09-23.

[2] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. SWE-Bench+: Enhanced Coding Benchmark for LLMs. arXiv:2410.06992 [cs.SE]

[3] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. arXiv:2505.20411 [cs.SE]

[4] Shawn A. Bohner and Robert S. Arnold (Eds.). 1996. *Software Change Impact Analysis.* IEEE Computer Society Press, Los Alamitos, CA.

[5] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. arXiv:2406.01304 [cs.SE]

[6] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers).* Association for Computational Linguistics, Vienna, Austria, 8697–8727. https://doi.org/10.18653/v1/2025.acl-long.426 Introduces the Loc-Bench benchmark. arXiv:2503.09089.

[7] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature Location in Source Code: A Taxonomy and Survey. *Journal of Software: Evolution and Process* 25, 1 (2013), 53–95. https://doi.org/10.1002/smr.567

[8] Kang He and Kaushik Roy. 2026. SWE-Adept: An LLM-Based Agentic Framework for Deep Codebase Analysis and Structured Issue Resolution. arXiv:2603.01327 [cs.SE]

[9] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. Issue Localization via LLM-Driven Iterative Code Graph Searching. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Preprint titled "CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching"; arXiv:2503.22424.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.06770. SWE-bench Lite is released with this benchmark.

[11] Amy J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks. *IEEE Transactions on Software Engineering* 32, 12 (Dec. 2006), 971–987. https://doi.org/10.1109/TSE.2006.116

[12] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. 2026. ContextBench: A Benchmark for Context Retrieval in Coding Agents. arXiv:2602.05892 [cs.SE]

[13] Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. 2025. The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason. arXiv:2506.12286 [cs.SE] ICSE-SEIP 2026.

[14] Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. 2025. GraphLocator: Graph-guided Causal Reasoning for Issue Localization. arXiv:2512.22469 [cs.SE]

[15] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2025. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), Industry Track*. https://doi.org/10.1145/3696630.3728549 arXiv:2406.01422 (v1 titled "How to Understand Whole Software Repository?").

[16] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-Level Code Graph. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.14684.

[17] Bowen Qin and Yi Xie. 2026. Agent Retrieval Bench: Evaluating Repository Context Retrieval for Coding Agents. arXiv:2607.24882 [cs.SE]

[18] Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. 2025. SweRank: Software Issue Localization with Code Ranking. arXiv:2505.07849 [cs.SE]

[19] Martin P. Robillard. 2008. Topology Analysis of Software Dependencies. *ACM Transactions on Software Engineering and Methodology* 17, 4 (Aug. 2008), 18:1–18:36. https://doi.org/10.1145/13487689.13487691

[20] Lintang Sutawika, Aditya Bharat Soni, Bharath Sriraam R R, Apurva Gandhi, Taha Yassine, Sanidhya Vijayvargiya, Yuchen Li, Xuhui Zhou, Yilin Zhang, Leander Melroy Maben, and Graham Neubig. 2026. CodeScout: An Effective Recipe for Reinforcement Learning of Code Search Agents. arXiv:2603.17829 [cs.SE]

[21] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (July 2025), FSE037. https://doi.org/10.1145/3715754 Agentless. arXiv:2407.01489.

[22] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. 2025. SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. In *Findings of the Association for Computational Linguistics: ACL 2025*. 1123–1139. https://doi.org/10.18653/v1/2025.findings-acl.62 arXiv:2501.05040.

[23] Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. 2025. Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. arXiv:2503.21710 [cs.SE] KGCompass. An earlier version was titled "KGCompass: Knowledge Graph Enhanced Repository-Level Software Repair".

[24] Annie T. T. Ying, Gail C. Murphy, Raymond Ng, and Mark C. Chu-Carroll. 2004. Predicting Source Code Changes by Mining Change History. *IEEE Transactions on Software Engineering* 30, 9 (Sept. 2004), 574–586. https://doi.org/10.1109/TSE.2004.52

[25] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2025. SWE-bench Goes Live! arXiv:2505.23419 [cs.SE] NeurIPS 2025 Datasets and Benchmarks Track (per the authors' repository).

[26] Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026. SWE-Explore: Benchmarking How Coding Agents Explore Repositories. arXiv:2606.07297 [cs.SE]

[27] Thomas Zimmermann, Peter Weißgerber, Stephan Diehl, and Andreas Zeller. 2004. Mining Version Histories to Guide Software Changes. In *Proceedings of the 26th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Edinburgh, United Kingdom, 563–572.

A SUPPORTING TABLES

Table 7. The two keys re-scored on the 702 pull requests that carry both, top ten. Δ denotes the change in rank relative to Table 3.

Method	Fam.	AUC	#	Δ
`oracle`	ref	.774	0	–
`rrf_pprpl_issue_path`	fus	.487	1	–
`rrf_hops_issue_path`	fus	.480	2	–
`rrf_hops_path`	fus	.457	3	–
`rrf_ppr_issue_path`	fus	.445	4	–
`path_issue`	lex	.435	5	–
`bm25_issue`	lex	.409	6	–
`rrf_ppr_issue`	fus	.394	7	−1
`bm25_seed`	lex	.393	8	+1
`hops_lines`	str	.389	9	–
`oracle`	ref	.779	0	–
`rrf_pprpl_issue_path`	fus	.470	1	−4
`rrf_hops_path`	fus	.456	2	−2
`rrf_hops_issue_path`	fus	.454	3	−4
`ppr_und_pl`	str	.436	4	+2
`hops_lines`	str	.425	5	+2
`rrf_ppr_issue_path`	fus	.421	6	−4
`ppr_out_pl`	str	.417	7	+6
`path_issue`	lex	.404	8	−6
`ppr_und`	str	.395	9	+1

Table 8. Both controls applied simultaneously: AUC and rank under three nested leakage strata, each computed on the pull requests within the stratum that carry both keys.

	no full path		no explicit mention		no path or stem	
Method	co_edited	symbol	co_edited	symbol	co_edited	symbol
`rrf_pprpl_issue_path`	.468 (1)	.456 (1)	.434 (1)	.430 (3)	.417 (2)	.414 (4)
`rrf_hops_path`	.441 (3)	.445 (2)	.410 (3)	.421 (5)	.392 (3)	.404 (6)
`ppr_out_pl`	.325 (16)	.419 (6)	.335 (15)	.427 (4)	.331 (15)	.423 (3)
`ppr_und_pl`	.374 (10)	.440 (4)	.378 (9)	.445 (1)	.373 (9)	.441 (1)
`hops_lines`	.390 (8)	.427 (5)	.394 (5)	.435 (2)	.387 (5)	.429 (2)
`path_issue`	.408 (5)	.383 (10)	.363 (12)	.343 (18)	.339 (13)	.320 (19)
`bm25_issue`	.397 (6)	.357 (14)	.384 (6)	.346 (15)	.385 (8)	.346 (16)
`pagerank`	.274 (20)	.356 (15)	.281 (20)	.360 (13)	.285 (20)	.361 (12)
pull requests	655	655	531	531	487	487

Table 9. Redaction by method: AUC on the original issue and the change under each arm, on the matched population within each stratum; * marks $p_{\text{Holm}} < 0.05$ within an arm.

	co_edited				symbol			
Method	AUC	names	paths	symbols	AUC	names	paths	symbols
issue names a key file as code (378 pull requests in 105 repositories)								
path_issue	.550	−0.110*	−0.148*	−0.157*	.498	−0.092*	−0.125*	−0.134*
bm25_issue	.452	−0.009*	−0.011*	−0.026*	.408	−0.006*	−0.008*	−0.021*
rrf_hops_path	.529	−0.072*	−0.097*	−0.105*	.515	−0.059*	−0.082*	−0.089*
rrf_pprpl_issue_path	.566	−0.061*	−0.084*	−0.096*	.537	−0.050*	−0.074*	−0.082*
issue names no key file as code (531 pull requests in 109 repositories)								
path_issue	.363	−0.000	−0.025*	−0.030*	.347	−0.002	−0.026*	−0.028*
bm25_issue	.376	−0.000	−0.003*	−0.006*	.341	−0.000	−0.002*	−0.004*
rrf_hops_path	.415	+0.001	−0.017*	−0.019*	.426	−0.001	−0.020*	−0.019*
rrf_pprpl_issue_path	.434	+0.002	−0.016*	−0.018*	.433	+0.000	−0.018*	−0.017*

Table 10. The dense retriever on the matched population: AUC and the rank that each ordering would obtain if inserted alone into the ranking of Table 3.

Ordering	co_edited	symbol
rrf_pprpl_issue_path_dense	.516 (1)	.480 (1)
rrf_pprpl_dense_path	.505 (1)	.488 (1)
rrf_pprpl_issue_path	.487 (1)	.470 (1)
rrf_pprpl_dense	.474 (3)	.476 (1)
dense_issue	.473 (3)	.422 (6)
path_issue	.435 (5)	.404 (8)
bm25_issue	.409 (6)	.367 (13)
dense_seed	.398 (7)	.364 (15)
bm25_seed	.393 (8)	.353 (16)

Table 11. AUC (and rank) of the leading orderings under three cost rules, matched population. *Stop*: the frozen rule. *Skip*: an overflowing file is skipped. *Files*: one unit per file.

	co_edited			symbol		
Ordering	stop	skip	files	stop	skip	files
rrf_pprpl_issue_path	.487 (1)	.519 (2)	.595 (6)	.470 (1)	.517 (1)	.571 (4)
rrf_ppr_issue_path	.445 (4)	.505 (3)	.649 (1)	.421 (6)	.496 (3)	.616 (1)
rrf_hops_issue_path	.480 (2)	.522 (1)	.631 (2)	.454 (3)	.509 (2)	.595 (2)
rrf_hops_path	.457 (3)	.479 (4)	.488 (9)	.456 (2)	.492 (4)	.483 (8)
hops_lines	.389 (9)	.399 (9)	.381 (14)	.425 (5)	.447 (8)	.410 (15)
ppr_und_pl	.372 (10)	.379 (13)	.268 (20)	.436 (4)	.448 (7)	.319 (19)
ppr_und	.358 (13)	.396 (10)	.497 (7)	.395 (9)	.456 (6)	.531 (7)
path_issue	.435 (5)	.456 (6)	.490 (8)	.404 (8)	.435 (10)	.452 (11)
bm25_issue	.409 (6)	.460 (5)	.615 (3)	.367 (13)	.418 (13)	.553 (5)

Table 12. Without a free seed: mean AUC per pull request. Suffix _ds: walked from the top file of the dense retriever; otherwise, walked from the BM25 guess. *Edited*: every edited file, including the true seeds; each key is split by whether the issue names an edited file as code.

	edited			symbol		
Ordering	all	named	not	all	named	not
`rrf_pprpl_dense_path_ds`	.516	.563	.469	.475	.503	.446
`rrf_pprpl_issue_path_ds`	.507	.553	.460	.461	.491	.431
`rrf_pprpl_dense_path`	.500	.553	.446	.458	.493	.423
`rrf_pprpl_issue_path`	.480	.533	.427	.440	.478	.403
`dense_issue`	.495	.521	.469	.417	.434	.400
issue-only fusion (picks the seed)	.471	.524	.417	.396	.436	.356
`path_issue`	.458	.531	.383	.397	.451	.342
`bm25_issue`	.417	.429	.405	.355	.369	.341
`ppr_und_pl_ds`	.409	.435	.383	.445	.456	.434
`ppr_und_pl`	.404	.448	.361	.429	.456	.402
`random`	.237	.239	.235	.240	.239	.240
oracle	.758	.763	.753	.760	.762	.758

Table 13. ContextBench tasks: AUC (and rank) under the human key, its read-only part (gold files that the fix does not edit; exploratory), and the two proxy keys.

Ordering	gold	read-only	co_edited	symbol
`rrf_hops_issue_path`	.379 (1)	.347 (1)	.436 (1)	.343 (3)
`rrf_pprpl_issue_path`	.370 (2)	.341 (2)	.423 (2)	.349 (2)
`rrf_hops_path`	.345 (4)	.317 (4)	.387 (3)	.338 (5)
`bm25_issue`	.283 (6)	.259 (9)	.333 (5)	.217 (16)
`path_issue`	.280 (8)	.248 (12)	.322 (6)	.228 (14)
`hops_lines`	.275 (9)	.274 (6)	.281 (9)	.309 (6)
`ppr_und_pl`	.261 (10)	.267 (7)	.261 (10)	.352 (1)
`ppr_out_pl`	.230 (14)	.249 (11)	.202 (15)	.339 (4)
`pagerank`	.180 (17)	.192 (17)	.159 (19)	.261 (11)
`random`	.103 (22)	.104 (22)	.104 (22)	.095 (22)
oracle	.694	.699	.739	.731

Table 14. 200 random 56/56 splits of the repositories: how often the criterion selected each fusion, how often that fusion was then the best on the held-out half, and its mean shortfall there.

		`co_edited`		`symbol`	
Method	selected	best	gap	best	gap
`rrf_pprpl_issue_path`	198/200	195/198	.002	198/198	.000
`rrf_hops_issue_path`	2/200	0/2	.016	0/2	.024

Table 15. Location of the key files relative to the seed, and size of the two-hop ball (union of the two keys, 2,412 tasks).

Key files at hop distance	1	2	3+	unreachable
co_edited	39.0%	49.1%	9.1%	2.8%
symbol	45.3%	41.4%	9.1%	4.2%
The two-hop ball: 68 files, 21,646 lines, 56% of the repository recall 87.5%, precision 3.4%, recall lift over a same-sized random sample 1.57×				
The union key: 3 files, 1,515 lines				

Table 16. How each key file is retrieved within the top k of the structural family (ppr_und, hops_lines) and the lexical family (bm25_issue, path_issue).

Source	k	both	structure only	words only	neither
co_edited	10	32.2%	11.4%	30.1%	26.4%
co_edited	20	50.9%	9.5%	24.4%	15.3%
co_edited	50	76.5%	6.1%	13.3%	4.1%
symbol	10	28.4%	23.4%	20.5%	27.7%
symbol	20	47.2%	20.0%	16.5%	16.2%
symbol	50	72.9%	11.5%	9.8%	5.9%

Table 17. Median number of lines read before half, and then all, of the key is covered.

	to half the key		to all of it	
Method	co_edited	symbol	co_edited	symbol
oracle	524	440	1,150	1,349
rrf_pprpl_issue_path	3,342	3,957	7,213	9,563
rrf_hops_path	3,962	3,715	8,073	9,636
ppr_out_pl	8,655	3,924	15,396	12,386
ppr_und_pl	6,155	3,483	11,644	11,303
hops_lines	5,620	4,080	11,454	11,582
path_issue	4,838	6,638	11,072	14,504
bm25_issue	5,012	7,451	10,926	16,699
random	14,557	14,713	21,026	22,643

Table 18. Share of pull requests in which coverage is completed within 8,000 lines.

Ordering	co_edited (%)	symbol (%)
oracle	97.5	95.9
rrf_hops_issue_path	58.5	49.3
rrf_pprpl_issue_path	56.5	49.1
rrf_hops_path	54.5	50.3
rrf_ppr_issue_path	53.4	45.0
bm25_issue	51.8	36.9
rrf_ppr_issue	50.6	44.6
bm25_seed	48.8	36.6
path_issue	48.4	36.9
hops_lines	48.1	51.0
ppr_und_pl	45.1	50.0
ppr_und	45.0	50.1
bm25_issue_pl	39.8	33.9
path_seed	39.6	36.3
bfs_und	36.6	40.1
ppr_out_pl	35.4	50.1
ppr_out	34.5	49.7
ppr_in	34.0	22.8
ppr_in_pl	33.9	22.3
pagerank	31.9	43.4
indegree	28.2	38.9
same_dir	27.8	24.2
random	15.3	14.3

Table 19. Mean AUC (rank) by quartile of repository size in graph nodes: Q1 ≤ 73, Q2 ≤ 131, Q3 ≤ 246, Q4 up to 2,487. Full corpus, no control applied.

	co_edited				symbol			
Method	Q1	Q2	Q3	Q4	Q1	Q2	Q3	Q4
rrf_pprpl_issue_path	.574 (1)	.487 (1)	.418 (1)	.369 (1)	.568 (2)	.476 (1)	.403 (4)	.375 (2)
rrf_hops_path	.552 (3)	.465 (3)	.397 (3)	.330 (4)	.562 (5)	.473 (2)	.409 (3)	.362 (4)
ppr_out_pl	.473 (15)	.359 (16)	.268 (15)	.168 (16)	.562 (4)	.450 (6)	.414 (2)	.372 (3)
ppr_und_pl	.497 (12)	.399 (8)	.316 (10)	.241 (11)	.568 (3)	.467 (3)	.423 (1)	.387 (1)
hops_lines	.517 (6)	.412 (6)	.329 (8)	.244 (10)	.569 (1)	.454 (5)	.402 (5)	.343 (7)
path_issue	.537 (5)	.437 (4)	.372 (5)	.302 (5)	.515 (13)	.416 (9)	.324 (12)	.257 (14)
bm25_issue	.516 (7)	.402 (7)	.334 (6)	.287 (6)	.476 (17)	.367 (17)	.273 (17)	.233 (16)
pagerank	.442 (20)	.299 (21)	.199 (20)	.113 (21)	.521 (10)	.370 (16)	.333 (11)	.286 (12)

B CORRECTION FAMILIES AND MIXED-MODEL ESTIMATES

Table 20. All Holm families. Paired families contain all 325 tests among twenty-six orderings (the twenty tested contenders, two issue-free twins and four names-redacted variants).

Tests	Population	Stratum	Keys	Tests per family
paired tests, pull request as unit	full	all	3	325
paired tests, pull request as unit	matched	all	3	325
paired tests, repository as unit	full	all	3	325
paired tests, repository as unit	full	explicit	2	325
paired tests, repository as unit	full	no explicit	2	325
paired tests, repository as unit	full	no full path	2	325
paired tests, repository as unit	full	no stem	2	325
paired tests, repository as unit	matched	all	3	325
paired tests, repository as unit	matched	explicit	2	325
paired tests, repository as unit	matched	no explicit	2	325
paired tests, repository as unit	matched	no full path	2	325
paired tests, repository as unit	matched	no stem	2	325
redaction, names arm	matched	by group	1	24
redaction, paths arm	matched	by group	1	24
redaction, symbols arm	matched	by group	1	24
mixed model, headline comparisons	both	several	1	41

Table 21. The comparisons on which claims rest, re-estimated as a linear mixed model of the per-pull-request difference $a - b$ with a random intercept per repository. Holm-corrected as a single family.

a	b	Key	Stratum	Pop.	PRs	Estimate [95% CI]	p_{Holm}
rrf_pprpl_issue_path	path_issue	co_edited	all	full	708	+0.053 [0.040,0.067]	$< 10^{-12}$
rrf_pprpl_issue_path	path_issue	co_edited	all	matched	702	+0.054 [0.040,0.067]	$< 10^{-12}$
rrf_pprpl_issue_path	path_issue	symbol	all	full	1156	+0.089 [0.075,0.104]	$< 10^{-30}$
rrf_pprpl_issue_path	path_issue	symbol	all	matched	702	+0.067 [0.054,0.079]	$< 10^{-23}$
ppr_out_pl	path_issue	co_edited	all	full	708	-0.105 [-0.125,-0.085]	$< 10^{-23}$
ppr_out_pl	path_issue	co_edited	all	matched	702	-0.106 [-0.126,-0.087]	$< 10^{-23}$
ppr_out_pl	path_issue	symbol	all	full	1156	+0.105 [0.082,0.127]	$< 10^{-17}$
ppr_out_pl	path_issue	symbol	all	matched	702	+0.012 [-0.012,0.037]	1
ppr_und_pl	path_issue	co_edited	all	full	708	-0.062 [-0.082,-0.042]	$< 10^{-7}$
ppr_und_pl	path_issue	co_edited	all	matched	702	-0.062 [-0.082,-0.043]	$< 10^{-7}$
ppr_und_pl	path_issue	symbol	all	full	1156	+0.103 [0.078,0.128]	$< 10^{-13}$
ppr_und_pl	path_issue	symbol	all	matched	702	+0.032 [0.007,0.056]	0.143
hops_lines	path_issue	co_edited	all	full	708	-0.044 [-0.064,-0.023]	$< 10^{-3}$
hops_lines	path_issue	co_edited	all	matched	702	-0.044 [-0.065,-0.024]	$< 10^{-3}$
hops_lines	path_issue	symbol	all	full	1156	+0.091 [0.067,0.115]	$< 10^{-11}$
hops_lines	path_issue	symbol	all	matched	702	+0.022 [-0.000,0.044]	0.610
rrf_pprpl_issue_path	ppr_out_pl	symbol	all	full	1156	-0.014 [-0.036,0.007]	1
rrf_pprpl_issue_path	ppr_out_pl	symbol	all	matched	702	+0.054 [0.032,0.076]	$< 10^{-4}$
rrf_pprpl_issue_path	ppr_out_pl	symbol	no explicit	full	871	-0.061 [-0.083,-0.040]	$< 10^{-6}$
rrf_pprpl_issue_path	ppr_out_pl	symbol	no explicit	matched	531	+0.000 [-0.024,0.024]	1
rrf_pprpl_issue_path	ppr_und_pl	symbol	all	full	1156	-0.013 [-0.036,0.009]	1
rrf_pprpl_issue_path	ppr_und_pl	symbol	all	matched	702	+0.035 [0.015,0.055]	0.008
rrf_pprpl_issue_path	ppr_und_pl	symbol	no explicit	full	871	-0.057 [-0.079,-0.035]	$< 10^{-5}$
rrf_pprpl_issue_path	ppr_und_pl	symbol	no explicit	matched	531	-0.016 [-0.037,0.005]	1
rrf_pprpl_issue_path	hops_lines	symbol	all	full	1156	-0.002 [-0.019,0.014]	1
rrf_pprpl_issue_path	hops_lines	symbol	all	matched	702	+0.045 [0.027,0.062]	$< 10^{-4}$
rrf_pprpl_issue_path	hops_lines	symbol	no explicit	full	871	-0.047 [-0.063,-0.032]	$< 10^{-7}$
rrf_pprpl_issue_path	hops_lines	symbol	no explicit	matched	531	-0.007 [-0.026,0.013]	1
hops_lines	path_issue	co_edited	no full path	matched	655	-0.016 [-0.037,0.004]	1
hops_lines	path_issue	co_edited	no explicit	matched	531	+0.034 [0.009,0.059]	0.103
hops_lines	path_issue	co_edited	no stem	matched	487	+0.050 [0.024,0.077]	0.004
ppr_out_pl	ppr_in_pl	symbol	all	full	1156	+0.215 [0.192,0.238]	$< 10^{-71}$
ppr_out_pl	ppr_in_pl	symbol	all	matched	702	+0.123 [0.104,0.143]	$< 10^{-33}$
rrf_pprpl_issue_path	rrf_pprpl_issue_path_rd	co_edited	explicit	matched	378	+0.063 [0.050,0.075]	$< 10^{-20}$
rrf_pprpl_issue_path	rrf_pprpl_issue_path_rd	co_edited	no explicit	matched	531	-0.002 [-0.005,0.001]	1
rrf_pprpl_issue_path	rrf_pprpl_issue_path_rd	symbol	explicit	matched	378	+0.049 [0.040,0.058]	$< 10^{-23}$
rrf_pprpl_issue_path	rrf_pprpl_issue_path_rd	symbol	no explicit	matched	531	-0.000 [-0.003,0.003]	1
rrf_pprpl_issue_path	rrf_pprpl_seedpath	co_edited	explicit	matched	378	+0.153 [0.130,0.176]	$< 10^{-36}$
rrf_pprpl_issue_path	rrf_pprpl_seedpath	co_edited	no explicit	matched	531	+0.020 [0.005,0.035]	0.133
rrf_pprpl_issue_path	rrf_pprpl_seedpath	symbol	explicit	matched	378	+0.116 [0.095,0.137]	$< 10^{-25}$
rrf_pprpl_issue_path	rrf_pprpl_seedpath	symbol	no explicit	matched	531	+0.012 [-0.003,0.027]	1

C A PROTOCOL FOR THE DOWNSTREAM TEST

Whether a context assembled by one of these orderings enables a model to fix the issue can only be determined by running models, which we have not done; we therefore specify the experiment here so that it can be run or refuted. *Corpus*: the 1,162 scored pull requests, each with a benchmark test specification. *Conditions*, each truncated to the same token budget of 16,000 or 64,000 tokens: (i) the seed alone; (ii) the seed plus `ppr_und_pl`; (iii) the seed plus `path_issue`; (iv) the seed plus the recommended fusion; (v) the seed plus `rrf_hops_path`; (vi) the seed plus the issue-free twin of (iv); (vii) the repository in a fixed canonical order; (viii) the seed plus the key files, as an upper reference. *Models*: two models of clearly different capability, with one API call per task and condition and no tools. *Outcome*: a task is considered resolved if the patch applies and the specified tests pass;

conditions are compared using a mixed-effects logistic regression with a random intercept per repository, with McNemar tests alongside and Holm correction across condition pairs. *Pilot*: 150 tasks on the weaker model; if (viii) resolves fewer than 10% or more than 90% of tasks, the budget is adjusted and the adjustment is reported. If (ii)–(vi) do not differ from (vii), retrieval quality does not transfer to repair at these budgets; if (iv) does not outperform (vi), the issue does not justify its inclusion downstream either.